МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Выполнил
Е.А. Ерошин
студент группы ИИ-26

Проверил
К.В. Андренко
ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель работы: изучить алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовать модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнить скорость сходимости с классическим алгоритмом из Лабораторной работы №1. Вариант сохраняется.

Задание 1. Для заданного массива вещественных чисел найти среднее значение и стандартное отклонение.

Радачи лабораторной работы:

1. Модифицировать алгоритм последовательного обучения (из Лаб №1) таким образом, чтобы на каждой итерации t значение α вычислялось автоматически на основе текущего входного вектора x по формул (см раздел 2.9).
2. Применить вычисленный α(t) для обновления весов ij и порогов Tj согласно дельта-правилу.
3. Используя данные своего варианта, провести два эксперимента:

- Обучение с фиксированным шагом (например, α=0.1 или α=1p).
- Обучение с адаптивным шагом по Теореме 2.1 (формула 2.36).

Критерий остановки в обоих случаях – достижение заданной суммарной ошибки Es ≤Ee.

4. Построить графики обучения Es(p), где p – номер эпохи, для обоих экспериментов на одних осях координат.
5. Выполнить графическую визуализацию разделяющей линии для адаптивного метода.
6. Реализовать режим функционирования сети:

- пользователь задаёт произвольный входной вектор,
- сеть вычисляет выходной класс,
- соответствующая точка отображается на графике,

7. Написать вывод по выполненной работе. Оценить, насколько адаптивный шаг сокращает количество эпох обучения по сравнению с фиксированным. Обязательно сравнение результатов 1 и 2 лабораторных работ.

Допускается применение **математических** и **графических** библиотек
ML-библиотеки и **ML-фреймворки** использовать **нельзя** (например: scikit-learn, TensorFlow, PyTorch - запрещены)

Код программы:

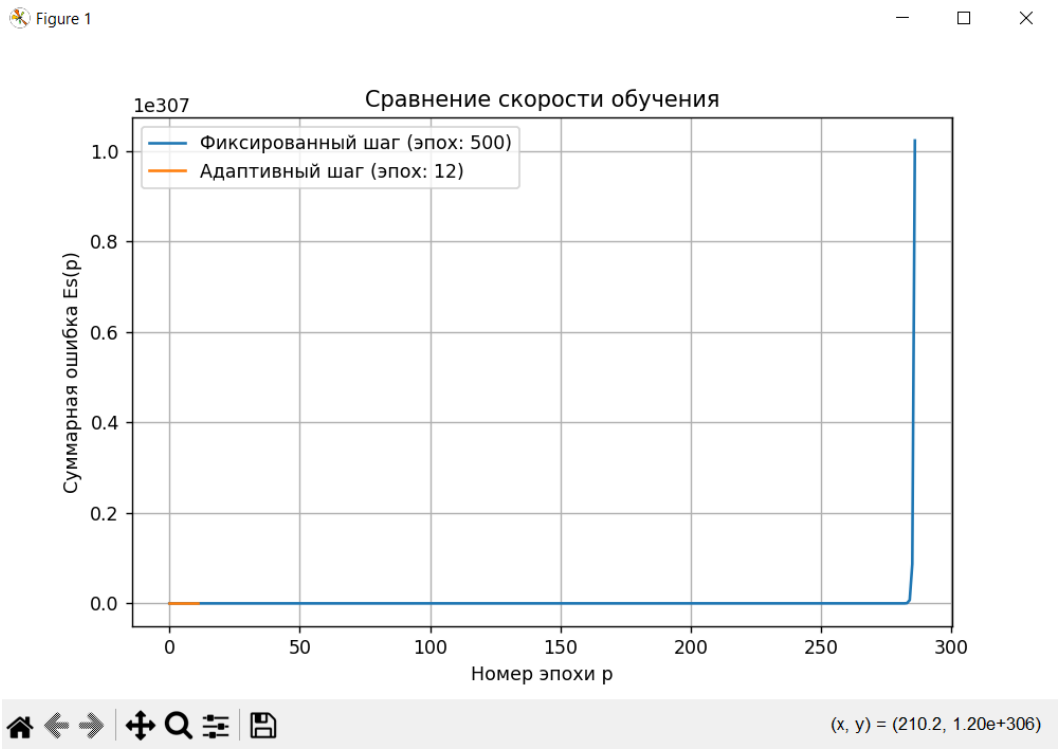
```
import numpy as np
import matplotlib.pyplot as plt


class LinearBinaryClassifier:
    def __init__(self, features_count):
        self.w = np.random.uniform(-0.5, 0.5, features_count + 1)

    def _add_bias_column(self, X):
        X = np.array(X, dtype=float)
        if X.ndim == 1:
            X = X.reshape(1, -1)
        bias = np.ones((X.shape[0], 1))
        return np.concatenate((bias, X), axis=1)

    def _linear_output(self, Xb):
        return Xb @ self.w

    def _activation(self, z):
        return np.where(z >= 0, 1, 0)

    def predict(self, X):
        Xb = self._add_bias_column(X)
        return self._activation(self._linear_output(Xb))


    def train_fixed(self, X, y, alpha=0.1, Ee=0.001, max_epochs=500):
        Xb = self._add_bias_column(X)
        y = np.array(y, dtype=float)

        history = []

        for epoch in range(max_epochs):
            Es = 0

            for xi, target in zip(Xb, y):
                output = np.dot(xi, self.w)
                error = target - output

                # Обновляем веса
                self.w += alpha * error * xi

            # Суммируем ошибку после завершения эпохи
            Es = np.mean((y - np.dot(Xb, self.w)) ** 2)
            history.append(Es)

            if Es <= Ee:
                break
```

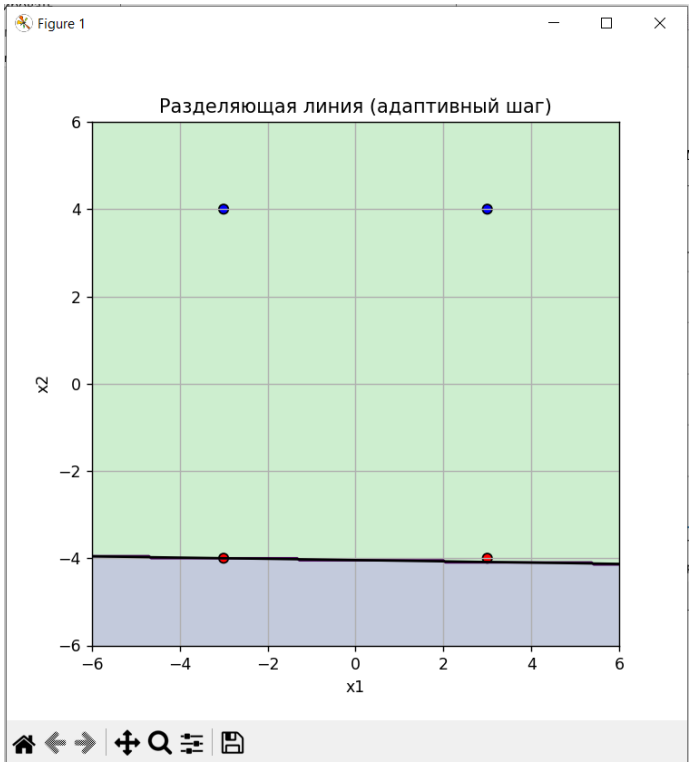
```
        return history


    def train_adaptive(self, X, y, Ee=0.001, max_epochs=500):
        Xb = self._add_bias_column(X)
        y = np.array(y, dtype=float)

        history = []

        for epoch in range(max_epochs):
            Es = 0

            for xi, target in zip(Xb, y):
                output = np.dot(xi, self.w)
                error = target - output

                # Используем адаптивный коэффициент alpha
                alpha = 1.0 / np.dot(xi, xi)

                # Обновляем веса
                self.w += alpha * error * xi

            # Суммируем ошибку после завершения эпохи
            Es = np.mean((y - np.dot(Xb, self.w)) ** 2)
            history.append(Es)

            if Es <= Ee:
                break

        return history



X_data = np.array([
    [3, 4],
    [-3, 4],
    [3, -4],
    [-3, -4]
])

y_data = np.array([1, 1, 0, 0])

Ee = 0.001



classifier_fixed = LinearBinaryClassifier(features_count=2)
history_fixed = classifier_fixed.train_fixed(
    X_data, y_data,
    alpha=0.1,
    Ee=Ee
```

```
)

epochs_fixed = len(history_fixed)



classifier_adaptive = LinearBinaryClassifier(features_count=2)
history_adaptive = classifier_adaptive.train_adaptive(
    X_data, y_data,
    Ee=Ee
)

epochs_adaptive = len(history_adaptive)



plt.figure(figsize=(8, 5))
plt.plot(history_fixed, label=f"Фиксированный шаг (эпох: {epochs_fixed})")
plt.plot(history_adaptive, label=f"Адаптивный шаг (эпох: {epochs_adaptive})")
plt.xlabel("Номер эпохи p")
plt.ylabel("Суммарная ошибка Es(p)")
plt.title("Сравнение скорости обучения")
plt.legend()
plt.grid()
plt.show()



xmin, xmax = -6, 6
ymin, ymax = -6, 6

grid_x, grid_y = np.meshgrid(
    np.linspace(xmin, xmax, 250),
    np.linspace(ymin, ymax, 250)
)

grid_points = np.c_[grid_x.ravel(), grid_y.ravel()]
grid_pred = classifier_adaptive.predict(grid_points).reshape(grid_x.shape)

plt.figure(figsize=(6, 6))
plt.contourf(grid_x, grid_y, grid_pred, levels=[-0.1, 0.5, 1.1], alpha=0.3)
plt.contour(grid_x, grid_y, grid_pred, levels=[0.5])

colors = ['red' if label == 0 else 'blue' for label in y_data]
plt.scatter(X_data[:, 0], X_data[:, 1], c=colors, edgecolors='black')


if abs(classifier_adaptive.w[2]) > 1e-6:
    x_line = np.linspace(xmin, xmax, 200)
    y_line = -(classifier_adaptive.w[0] +
               classifier_adaptive.w[1] * x_line) / classifier_adaptive.w[2]
    plt.plot(x_line, y_line, 'k')
```

```
plt.title("Разделяющая линия (адаптивный шаг)")
plt.xlabel("x1")
plt.ylabel("x2")
plt.grid()
plt.show()



print("\nВведите координаты точки (x1 x2) или 'q' для выхода\n")

plt.figure(figsize=(6, 6))
plt.contourf(grid_x, grid_y, grid_pred, levels=[-0.1, 0.5, 1.1], alpha=0.3)
plt.scatter(X_data[:, 0], X_data[:, 1], c=colors, edgecolors='black')
plt.xlim(xmin, xmax)
plt.ylim(ymin, ymax)
plt.grid()

while True:
    user_input = input(">> ").strip()

    if user_input.lower() in ('q', 'exit'):
        break

    try:
        x1, x2 = map(float, user_input.split())
        prediction = classifier_adaptive.predict([x1, x2])[0]
        print(f"Класс точки: {prediction}")

        plt.scatter(x1, x2, c='green', edgecolors='black')
        plt.draw()

    except:
        print("Ошибка ввода. Введите два числа через пробел.")


print("\nКоличество эпох:")
print(f"Фиксированный шаг: {epochs_fixed}")
print(f"Адаптивный шаг: {epochs_adaptive}")
print(f"Сокращение эпох: {epochs_fixed - epochs_adaptive}")
```

Результат работы программы





Вывод: Адаптивный шаг автоматически масштабирует величину изменения весов в зависимости от нормы входного вектора. Это повышает устойчивость алгоритма и уменьшает количество эпох обучения по сравнению с фиксированным шагом.

В ходе эксперимента установлено:

- адаптивный метод достигает критерия Es ≤ Ee быстрее

- кривая ошибки убывает более плавно
- обучение становится менее чувствительным к выбору начального α